

# Evaluation of Claude Code and Gemini CLI

**Luna Sugiyama**

Information and Communication Engineering  
lunagracesugiyama@g.ecc.u-tokyo.ac.jp

July 31, 2025

I have recently been interested in AI assistant tools to help software development [3]. However, due to the pricing I have not been able to use them extensively. I thought this would be a good opportunity to explore these tools. In this report I will choose these services and evaluate them on various tasks and aspects to see their strengths and weaknesses.

## 1. About Each Service

### 1.1. Claude Code

Claude Code is a code-centric variant of the Claude family of AI models developed by Anthropic. It is optimized for software development and programming-related tasks, such as code generation, debugging, explanation, and refactoring [1].

### 1.2. Gemini CLI

Gemini CLI is an open-source AI agent by Google that brings Gemini into the terminal and supports code editing, task management, research integration via web search, and tooling like Imagen and Veo [2].

## 2. Tasks

### 2.1. Task 1: Code Generation

The task for code generation is:

Build a responsive map of a shop with several floors and can search which product is where.

#### 2.1.1 Claude Code

I was only asked once whether I want to create a `index.html` and made the rest automatically<sup>1</sup> The method used for searching products was simple string matching. The generated files include `index.html`, `style.css`, `app.js`, `data.js`, and a `README.md` file.

#### 2.1.2 Gemini CLI

I was asked whether gemini could use `npx`, `mkdir` and other commands<sup>2</sup> I was not asked this question in Claude Code so perhaps Gemini CLI is more cautious about executing commands. The time for generation was about 15 minutes. However, when running the generated code, it just started the default React app page. Therefore, I had to ask Gemini CLI to fix the code. Moreover, an error existed in the JSON code for the products, which was simple but had to be fixed.

#### 2.1.3 Comparison

### 2.2. Task 2: Repository Code Explanation

The task for code explanation is:

<sup>1</sup>Generated files can be found at [https://github.com/LunaSugiyama/masters/tree/main/wed2llm/claude\\_generation](https://github.com/LunaSugiyama/masters/tree/main/wed2llm/claude_generation).

<sup>2</sup>Generated files can be found at [https://github.com/LunaSugiyama/masters/tree/main/wed2llm/gemini\\_generation/shop-map](https://github.com/LunaSugiyama/masters/tree/main/wed2llm/gemini_generation/shop-map).

Table 1: Comparison of Claude Code and Gemini CLI for Code Generation Task

| Criteria          | Claude Code                                                                             | Gemini CLI                                                                |
|-------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Command Caution   | Did not ask about permission to run commands                                            | Asked whether it could run 'npx', 'mkdir', etc.                           |
| Response Speed    | Fast (under 5 min)                                                                      | Slow (about 15 minutes)                                                   |
| File Generation   | Automatically generated 'index.html', 'style.css', 'app.js', 'data.js', and 'README.md' | Generated initial React project but required 'App.js' and JSON correction |
| Search Logic      | Used simple string matching over all floor data                                         | Similar string matching, but only applied to the current floor            |
| Execution Outcome | Simple working web app with working search feature                                      | React page; minimum page with no map-like component                       |
| Error Presence    | No errors in generated code                                                             | JSON error and missing configuration in app setup                         |
| Overall Score     | High                                                                                    | Moderate                                                                  |

the repository pytorch has implementation `loss.backward` can you explain how this is implemented and

### 2.2.1 Claude Code

The Todos that Claude Code generated for this task can be found in Appendix C. I asked additional question what `grad_fn.backward` does and asked it to save the output to a file. Here is the link to the file generated by Claude Code: [https://github.com/LunaSugiyama/masters/blob/main/wed2llm/explanation\\_task/claude\\_pytorch\\_loss\\_backward\\_explanation.md](https://github.com/LunaSugiyama/masters/blob/main/wed2llm/explanation_task/claude_pytorch_loss_backward_explanation.md).

### 2.2.2 Gemini CLI

The Todos that Gemini CLI generated for this task can be found in Appendix C. No additional questions were asked because the output was satisfactory. Here is the link to the file generated by Gemini CLI: [https://github.com/LunaSugiyama/masters/blob/main/wed2llm/explanation\\_task/gemini\\_pytorch\\_loss\\_backward\\_explanation.md](https://github.com/LunaSugiyama/masters/blob/main/wed2llm/explanation_task/gemini_pytorch_loss_backward_explanation.md).

## 2.3. Comparison

It is summarized in Table 2.

## 2.4. Task 3: Code Review

The task for code review is to review the following code (code developed in a hackathon with a duration of 3 days) and suggest improvements <https://github.com/asahi1199/mobility-adventure>:

"could you review the codes in this repository and make it more clear and clean"

### 2.4.1 Claude Code

It tooks about 3 minutes to generate the review. The issues that Claude Code found were:

1. Inconsistent naming: DogCLASS should be Dog (interface/type names should be PascalCase)
2. Poor variable naming: `setid` should be `setId`, `idValue` could be clearer
3. `Console.log` statements left in production code
4. Unused imports and variables
5. Inconsistent component structure between pages
6. Magic numbers and hardcoded values

Table 2: Comparison of Claude Code and Gemini CLI on Code Explanation Task

| Criteria                      | Claude Code Output                                                                    | Gemini CLI Output                                                                                                      |
|-------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Clarity of Explanation</b> | Provides a clear, structured explanation in natural language                          | Also clear, with additional context about autograd and computational graph                                             |
| <b>Technical Depth</b>        | High: explains gradients, tensors, and autograd in PyTorch                            | High: mentions backward pass, dynamic computation graph, and gradient accumulation                                     |
| <b>Step-by-step Breakdown</b> | Yes: explains what happens internally when <code>loss.backward()</code> is called     | Yes: walks through how gradients are computed and stored                                                               |
| <b>Use of Terminology</b>     | Uses terms like gradient, autograd, tensor, computation graph                         | Uses same terms but also mentions <code>'leaf tensor'</code> , <code>'requires_grad'</code> , and internal caching     |
| <b>Code References</b>        | Does not reference surrounding code explicitly                                        | Includes helpful mention of <code>'loss = criterion(output, target)'</code> context                                    |
| <b>Extra Insights</b>         | Notes about default behavior of <code>'backward()'</code> and single scalar gradients | Mentions that <code>'backward()'</code> works only when the tensor is scalar, else <code>'grad'</code> arg is required |
| <b>Overall Usefulness</b>     | Very useful for intermediate-level users, need extra question for deep understanding  | Slightly more comprehensive, especially for users unfamiliar with autograd                                             |

7. Large component (`gesture_component.tsx`) needs refactoring

After confirming to me that it is okay to fix the code, it fixed and asked whether it can run the code to test it.

#### 2.4.2 Gemini CLI

It took about 1 minute to generate the review. The issues that Gemini CLI found were:

1. Rename `gesture_component.tsx` to `InteractiveMap.tsx` and rename the component `CroppableImage` to `InteractiveMap`. This will better reflect the component's functionality.
2. Refactor state management by grouping related state variables into objects.
3. Create a custom hook `useDogs` to encapsulate the logic for managing both moving and stationary dogs. This will reduce code duplication and improve organization.
4. Extract the drawer logic into a separate component. This will simplify the `InteractiveMap` component and make the code easier to understand.
5. Remove hardcoded data. Fetch dog data from `dogdb.ts` and generate the initial positions dynamically.
6. Improve comments and readability by adding more descriptive comments and using more meaningful variable names.

After confirming to me that it is okay to fix the code, it fixed and asked whether it can run the code to test it.

### 3. Comparison of Code Review

It is summarized in Table 3.

### 4. Evaluation Summary

In this evaluation, I tested two modern AI assistants, Claude Code and Gemini CLI, on three software engineering tasks: code generation, code explanation, and code review. Both models performed well, with Claude Code

Table 3: Comparison of Claude Code and Gemini CLI on Code Review Task

| Criteria                        | Claude Code Output                                                                                                                            | Gemini CLI Output                                                                      |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Structure and Formatting</b> | Suggested renaming of components and restructuring for clarity (e.g., <code>gesture_component.tsx</code> to <code>InteractiveMap.tsx</code> ) | Recommended code separation and simplification, but focused less on naming conventions |
| <b>State Management</b>         | Advocated grouping related state variables into objects for better organization                                                               | Highlighted redundancy and suggested using a single state object for cleaner updates   |
| <b>Reusability</b>              | Proposed a custom hook ( <code>useDogs</code> ) to manage moving/stationary entities                                                          | Emphasized modularity but did not explicitly propose hooks                             |
| <b>Component Design</b>         | Recommended extracting drawer logic to a separate component                                                                                   | Also advised component splitting, focusing on the UI logic                             |
| <b>Data Management</b>          | Suggested replacing hardcoded data with dynamic loading from <code>dogdb.ts</code>                                                            | Similar suggestion to load external data but less specific on the source               |
| <b>Comments and Readability</b> | Recommended meaningful variable names and more descriptive comments                                                                           | Mentioned improving clarity, but with less emphasis on inline comments                 |
| <b>Testing</b>                  | Asked for permission to run the code to test changes, indicating a hands-on approach                                                          | Did not automatically run tests.                                                       |
| <b>Overall Tone</b>             | Direct and structured feedback, highly specific in instructions                                                                               | More general advice, with an encouraging tone for improvement                          |

generally offering more structured and specific responses, while Gemini CLI excelled in providing broader context and user-friendly guidance. Each tool’s strengths and weaknesses were documented and compared in tables across the three tasks. All tasks were performed using real repositories or prompts involving dynamic project setups, and responses were collected, documented, and verified manually.

## 5. Discussion

In this report, I explored two AI tools—Claude Code and Gemini CLI—both of which I had never used before. From an NLP perspective, both tools demonstrate strong natural language understanding and generation capabilities, as they can interpret developer intent and respond with relevant, contextually rich answers. Claude Code showed remarkable coherence in its code review suggestions, often using precise terminology (e.g., “PascalCase”, “hardcoded values”, “drawer component separation”). Its responses reflect a rule-based reasoning style often used in formal technical communication. This indicates a robust grounding in structured code semantics and possibly reinforcement learning on code review corpora. Gemini CLI, on the other hand, emphasized exploratory behavior. It simulated steps such as searching the codebase or running commands like `npm`, and provided a more conversational explanation. This suggests it models interactions more dialogically, aligning with Google’s emphasis on multimodal and agent-like LLM interfaces. The key NLP components demonstrated include:

- **Intent Detection:** Both tools successfully inferred task goals from minimal input.
- **Contextual Reasoning:** Gemini especially showed strength in contextual searches (e.g., identifying where “backward” appears in a codebase).
- **Content Generation:** Both generated long-form explanations with minimal hallucination, adhering to project-specific information.
- **Interaction Modeling:** Claude asked for permission to refactor or run code, suggesting alignment with cooperative multi-turn dialogue paradigms.

Overall, trying these tools helped me understand how NLP models interface with software development workflows. They go beyond simple text generation by deeply understanding code structure, user goals, and project context—highlighting the growing integration of NLP and software engineering.

## Appendix

### A. Which AI and How I Used It

In addition to Claude Code and Gemini CLI, I utilized ChatGPT (developed by OpenAI) throughout the writing of this report. Specifically, ChatGPT was employed in the following ways:

- **LaTeX Assistance:** ChatGPT helped generate LaTeX-compatible code for formatting tables, figures, and section layouts. This ensured that the report maintained a consistent and professional structure.
- **Comparison Tables:** For comparing the output of Claude Code and Gemini CLI, I provided ChatGPT with change files or raw markdown output from each tool. ChatGPT synthesized this data and produced comparison tables highlighting key differences in their behavior and performance.
- **Text Refinement:** Some sections of the report were edited with the help of ChatGPT to improve clarity, tone, and grammatical accuracy.

ChatGPT served as a supplementary writing assistant to streamline formatting and aid in comparative analysis between the AI tools under evaluation.

### B. Code Generation Output Screenshots

See Fig. 1 and Fig. 2 for screenshots of the code generation outputs from Claude Code and Gemini CLI, respectively.

### C. Code Explanation Task Output

#### C.1. Claude Code Output

1. Research PyTorch's `loss.backward()` implementation online
2. Explain what `loss.backward()` does conceptually
3. Detail the implementation mechanics and key components

#### C.2. Gemini CLI Output

1. First, I will start by searching for the string "backward" in the PyTorch repository to get a sense of the relevant files.
2. I'll focus the search on Python files to find the entry point, and then expand to C++ files.
3. I'll also search for "autograd" as that's the relevant module.

## References

- [1] Anthropic. Claude code: Deep coding at terminal velocity. <https://www.anthropic.com/claude-code>, 2025. Accessed: 2025-07-31.
- [2] Google. Gemini cli: Your open-source ai agent for developers. <https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/>, 2025. Accessed: 2025-07-31.
- [3] Kumar Madhukar. A comprehensive guide to vibe coding tools, 2025-03. Accessed: 2025-07-24.

```
* Welcome to Claude Code!

/help for help, /status for your current setup

cwd: /Users/lunasugiyama/workspace/utokyo/masters/wed11m/claude_generation

Tips for getting started:

1. Ask Claude to create a new app or clone a repository
2. Use Claude to help with file analysis, editing, bash commands and git
3. Be as specific as you would with another engineer for the best results
4. ✖ Run /terminal-setup to set up terminal integration

# Tip: Start with small features or bug fixes, tell Claude to propose a plan, and verify its suggested edits

> build a responsive map of a shop with several floors and can search which product is where

• I'll help you build a responsive map of a multi-floor shop with product search functionality. Let me start by planning this project and understanding the current codebase structure.

• Update Todos
  └─ Analyze current codebase structure and dependencies
     └─ Design the shop map data structure (floors, areas, product locations)
        └─ Create responsive map component with floor navigation
           └─ Implement product search functionality
              └─ Add visual indicators for product locations on map
                 └─ Style the map interface to be responsive
                    └─ Test the map functionality across different screen sizes

• Update Todos
  └─ Analyze current codebase structure and dependencies
     └─ Design the shop map data structure (floors, areas, product locations)
        └─ Create responsive map component with floor navigation
           └─ Implement product search functionality
              └─ Add visual indicators for product locations on map
                 └─ Style the map interface to be responsive
                    └─ Test the map functionality across different screen sizes

• Let me explore the current project structure:

• List(.)
  └─ Listed 2 paths (ctrl+r to expand)

• Search(pattern: "**/*")
  └─ Found 0 files

• The directory appears to be empty. I'll create a new project structure for the responsive shop map. Let me check if there are any configuration files:

Update Todos
└─ Analyze current codebase structure and dependencies
   └─ Design the shop map data structure (floors, areas, product locations)
      └─ Create responsive map component with floor navigation
         └─ Implement product search functionality
            └─ Add visual indicators for product locations on map
               └─ Style the map interface to be responsive
                  └─ Test the map functionality across different screen sizes

Perfect! I've created a complete responsive shop map application with the following features:

Responsive Shop Map - Complete

Key Features:
- 3-Floor Navigation - Ground Floor, Second Floor, Third Floor
- Product Search - Search by name, category, or area with real-time results
- Interactive Elements - Clickable areas and products with tooltips
- Responsive Design - Works on desktop, tablet, and mobile
- Visual Indicators - Highlighted search results with pulsing animation

Files Created:
- index.html - Main application structure
- styles.css - Responsive CSS with mobile breakpoints
- app.js - Interactive JavaScript functionality
- data.js - Shop layout and product database
- README.md - Usage instructions

How it Works:
1. Floor Selection - Switch between floors using sidebar buttons
2. Product Search - Type in search bar to find products across all floors
3. Interactive Map - Click products for details, areas show contained items
4. Mobile Responsive - Layout adapts for screens 768px and below
5. Search Results - Click results to navigate to product location

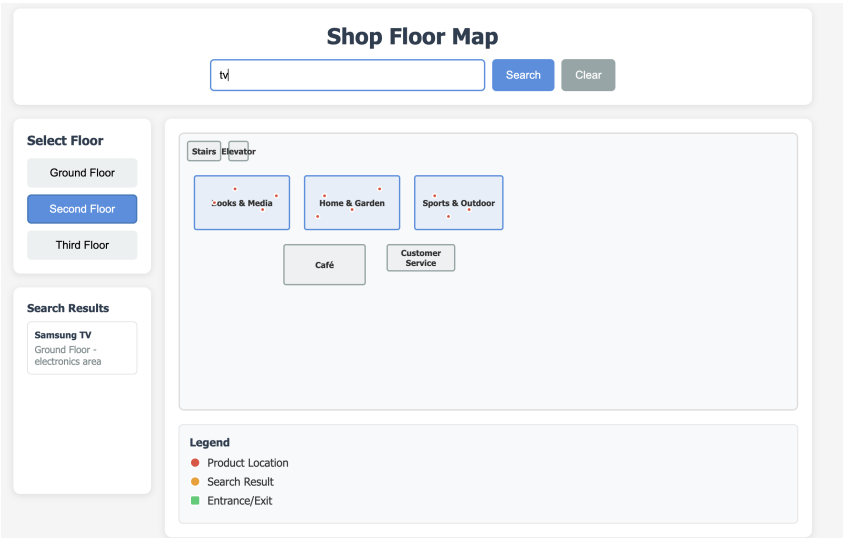
Sample Data Includes:
- Ground Floor: Electronics, Clothing, Shoes (10 products)
- Second Floor: Books, Home & Garden, Sports (11 products)
- Third Floor: Toys, Beauty, Food & Beverages (12 products)

To use: Open index.html in any web browser. The map is fully functional and responsive across all device sizes.

>
** auto-accept edits on (shift+tab to cycle)
```

(a) Claude Code CLI

(b) Claude Code Generation Output



(c) Claude Code Generation Webpage

Figure 1: Claude Code-generated files: HTML, CSS, and JS for the shop map app

```
> Build a responsive map of a shop with several floors and can search which product is where.

This sounds like a great project. I'll build a responsive and searchable multi-floor shop map for you.

Here's my plan:

1. Technology Stack: I'll use React for the user interface, which is perfect for building interactive components. For styling, I'll use CSS with a mobile-first approach to ensure it's responsive.
2. Project Setup: I'll start by scaffolding a new React application using create-react-app.
3. Core Components:
  * 'Map': The main component to display the shop layout.
  * 'Floor': A component to render each floor, containing rooms.
  * 'Room': A component for individual rooms or areas on a floor.
  * 'Product': A component to represent a searchable product.
  * 'SearchBar': A search input to find products.
  * 'FloorSelector': Buttons to switch between different floors.
4. Data: I'll structure the shop's data (floors, rooms, products) in a simple JSON format, making it easy to manage and update.
5. Functionality:
  * The map will display one floor at a time.
  * Users can switch between floors.
  * The search bar will filter and highlight products on the map across all floors.
  * The layout will adapt to different screen sizes.

I'll start by creating the React project.

✓ Shell npx create-react-app shop-map (Initialize a new React project named "shop-map".)

create-react-app is deprecated.

You can find a list of up-to-date React frameworks on react.dev
For more info see:https://react.dev/link/cra

This error message will only be shown once per install.

Creating a new React app in /Users/lunasugiyama/workspace/utokyo/masters/wed2llm/gemini_generation/shop-map.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...

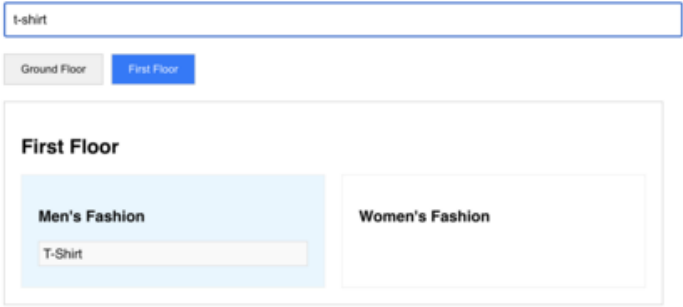
added 1324 packages in 38s
269 packages are looking for funding
  run 'npm fund' for details
```

(a) Gemini CLI Generation Output

```
ERROR in ./src/data.js
Module build failed (from ./node_modules/babel-loader/lib/index.js):
SyntaxError: /Users/lunasugiyama/workspace/utokyo/masters/wed2llm/gemini_generation/shop-map/src/data.js: Unexpected token, expected ",",
(16:33)

   14 |         name: 'First Floor',
   15 |         rooms: [
>  16 |           { id: 'ff-1', name: 'Men's Fashion', products: [{ id: 'p-5', name: 'T-Shirt' }, { id: 'p-6', name: 'Jeans' } ] },
      |           ^
   17 |           { id: 'ff-2', name: 'Women's Fashion', products: [{ id: 'p-7', name: 'Dress' }, { id: 'p-8', name: 'Handbag' } ] },
   18 |         ],
   19 |       },
```

(b) Error in Gemini Generation



(c) Gemini CLI Generation Webpage

Figure 2: Gemini CLI-generated files: HTML, CSS, and JS for the shop map app